

# What optimisations are worthwhile for filter, partition, and tokenisation on GPU?

**Lilje Henrich Due**<sup>1</sup>   Troels Henriksen<sup>1</sup>   Cosmin Eugen Oancea<sup>1</sup>

<sup>1</sup>Department of Computer Science, University of Copenhagen

August 20th, 2026

Contact: [due@di.ku.dk](mailto:due@di.ku.dk)

# Motivation

- Primitive Operations
  - Filter
  - Partition
- Non-uniform Data-Parallelism
  - Hash Maps
  - Quick Sort
  - Segmented Reduce
- Pointer Structures
  - Expression Evaluation
  - Union-Find
  - List Ranking

# Filter written in Futhark

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : [n]a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1)  
              flags  
              offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array

# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : [n]a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1)  
    flags  
    offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array

# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as                                [t, f, t, f, t, f]  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1)  
              flags  
              offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array

# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as                                [t, f, t, f, t, f]  
  let offsets = scan (+) 0 (map bool.i64 flags)       [1, 1, 2, 2, 3, 3]  
  let is = map2 (\f o -> if f then o - 1 else -1)  
              flags  
              offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array

# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as                                [t, f, t, f, t, f]  
  let offsets = scan (+) 0 (map bool.i64 flags)       [1, 1, 2, 2, 3, 3]  
  let is = map2 (\f o -> if f then o - 1 else -1)    [0, -1, 1, -1, 2, -1]  
    flags  
    offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array

# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as                                [t, f, t, f, t, f]  
  let offsets = scan (+) 0 (map bool.i64 flags)       [1, 1, 2, 2, 3, 3]  
  let is = map2 (\f o -> if f then o - 1 else -1)    [0, -1, 1, -1, 2, -1]  
    flags                                              [0, -1, 1, -1, 2, -1]  
    offsets                                           [0, -1, 1, -1, 2, -1]  
  let result = scatter (copy as) is as                [0, 2, 4, 3, 4, 5]  
  in take offsets[n - 1] result
```

\* copied array is actually a scratch array



# Filter written in Futhark

**Example using an is even predicate:** `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as                                [t, f, t, f, t, f]  
  let offsets = scan (+) 0 (map bool.i64 flags)       [1, 1, 2, 2, 3, 3]  
  let is = map2 (\f o -> if f then o - 1 else -1)    [0, -1, 1, -1, 2, -1]  
    flags                                             [0, 2, 4, 3, 4, 5]  
    offsets                                           [0, 2, 4]  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

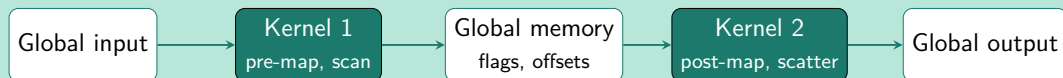
\* copied array is actually a scratch array

# Filter written in Futhark

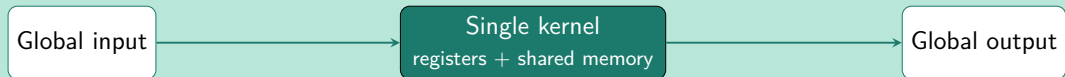
```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```

# Filter written in Futhark

```
def filter [n] 'a (p: a -> bool) (as: [n]a) : []a =  
  let flags = map p as  
  let offsets = scan (+) 0 (map bool.i64 flags)  
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets  
  let result = scatter (copy as) is as  
  in take offsets[n - 1] result
```



# Filter written in CUDA



- Streaming Scan.
- Avoid global memory round-trip.
- Not the only optimisation.

- Many hand-written CUDA kernels.
- Baseline is a two-kernel implementation mirroring compiler-generated code:
  - Kernel 1: map + scan
  - Kernel 2: map + scatter
- Single-kernel implementations are compared against this baseline.
- Vary optimisations.

# Hand-Written CUDA Prototypes of Filter

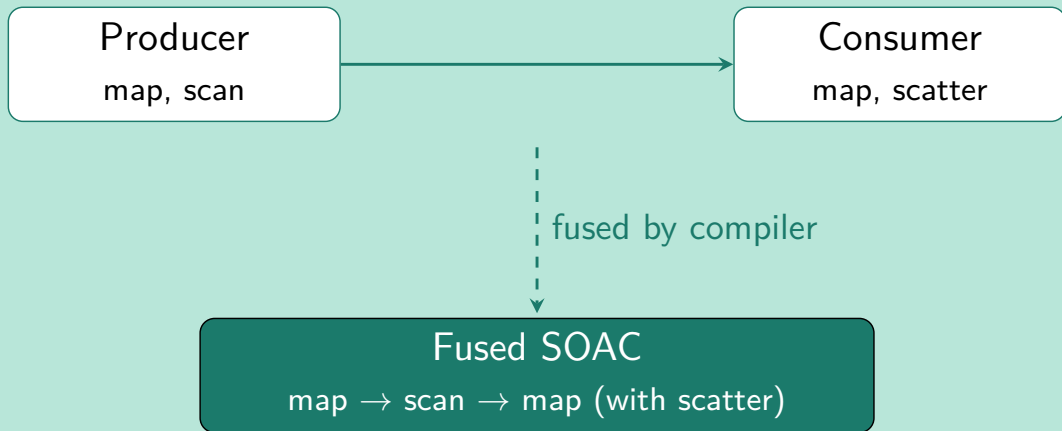
Relative speedup over **2K** baseline (higher is better)

Datasets vary by % of elements written: **Empty** (0%) → **Full** (100%)

| Implementation       | Full | Dense | Moderate | Sparse | Empty |
|----------------------|------|-------|----------|--------|-------|
| <b>2K</b> (baseline) | 1×   | 1×    | 1×       | 1×     | 1×    |
| <b>1K</b>            | 3.5× | 3.7×  | 4×       | 4.4×   | 4.9×  |
| <b>B</b>             | 3.5× | 3.7×  | 4.3×     | 4.9×   | 5.7×  |
| <b>CW</b>            | 3.4× | 3.6×  | 3.8×     | 4.2×   | 5.7×  |
| <b>FSMW</b>          | 3.5× | 3.8×  | 4.2×     | 5×     | 5.9×  |

**2K**: two-kernel baseline   **1K**: single kernel   **B**: bitsets in registers   **CW**: coalesced writing to global memory   **FSMW**: fewer shared memory writes

# Fusion of IR



No source language fused primitive — only `map`, `scan`, and `scatter`, combined by fusion.

```
scanscatter  $f \ (\oplus) \ g$  as  $ds =$   
  let  $(bs, gs) = (\text{unzip} \circ \text{map } f) \text{ as}$   
  let  $bs' = \text{scan } (\oplus) \ bs$   
  let  $(is, vs, ss) = (\text{unzip}_3 \circ \text{map}_2 \ g) \ bs' \ gs$   
  let  $ds' = \text{scatter } ds \ is \ vs$   
  in  $(ds', ss)$ 
```



# Accumulators

- Write-only array view; updates accumulate in place.
- Linear types.
- $[](\text{acc } \tau) \cong \text{acc } \tau$  — so `map` can take and return accumulators.

$$\begin{aligned}\text{with\_acc} &: []\tau \rightarrow (\text{acc } \tau \rightarrow \text{acc } \tau) \rightarrow []\tau \\ \text{update\_acc} &: \text{acc } \tau \rightarrow \mathbf{int} \rightarrow \tau \rightarrow \text{acc } \tau\end{aligned}$$

Scatter, expressed via accumulators:

$$\text{with\_acc } (\lambda \text{acc} \rightarrow \text{map}_2 \ (\lambda i \ v \rightarrow \text{update\_acc } \text{acc } i \ v) \text{ } \text{is } \text{vs}) \text{ } \text{ds}$$

# Generalization of IR

```
maposcanomap  $f$  ( $\oplus$ )  $g$   $as$  =  
  let  $(bs, gs) = (\text{unzip} \circ \text{map } f) as$   
  let  $bs' = \text{scan } (\oplus) bs$   
  in  $\text{map}_2 g bs' gs$ 
```

# Generalization of IR

```
maposcanomap  $f$  ( $\oplus$ )  $g$   $as$  =  
  let  $(bs, gs) = (\text{unzip} \circ \text{map } f) as$   
  let  $bs' = \text{scan } (\oplus) bs$   
  in  $\text{map}_2 g bs' gs$ 
```

```
 $\text{scan } \oplus \equiv \text{maposcanomap } (\lambda a \rightarrow (fa, ())) (\oplus) (\lambda a \_ \rightarrow a) as$ 
```

# Generalization of IR

```
maposcanomap  $f$  ( $\oplus$ )  $g$   $as$  =  
  let  $(bs, gs) = (\text{unzip} \circ \text{map } f) as$   
  let  $bs' = \text{scan } (\oplus) bs$   
  in  $\text{map}_2 g bs' gs$ 
```

```
 $\text{scan } \oplus \equiv \text{maposcanomap } (\lambda a \rightarrow (fa, ())) (\oplus) (\lambda a \_ \rightarrow a) as$ 
```

```
 $\text{scanomap } f \oplus \equiv \text{maposcanomap } f (\oplus) (\lambda a b \rightarrow (a, b)) as$ 
```

## Results: Filter

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| Old Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| New Futhark | 3.5× | 3.6×  | 4×       | 4.4×   | 4.5×  |

## Results: Filter

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| Old Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| New Futhark | 3.5× | 3.6×  | 4×       | 4.4×   | 4.5×  |

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| New Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| CUB         | 1×   | 1.1×  | 1.1×     | 1.2×   | 1.2×  |

## Results: Partition

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| Old Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| New Futhark | 2.5× | 2.6×  | 2.6×     | 2.6×   | 2.3×  |

## Results: Partition

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| Old Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| New Futhark | 2.5× | 2.6×  | 2.6×     | 2.6×   | 2.3×  |

| Method      | Full | Dense | Moderate | Sparse | Empty |
|-------------|------|-------|----------|--------|-------|
| New Futhark | 1×   | 1×    | 1×       | 1×     | 1×    |
| CUB         | 1.4× | 1.4×  | 1.4×     | 1.4×   | 1.4×  |



## Results: Tokenizer

| Method      | Dense | Moderate | Sparse |
|-------------|-------|----------|--------|
| Old Futhark | 1×    | 1×       | 1×     |
| New Futhark | 1.6×  | 1.8×     | 2×     |

- Simulates DFA.
- Determines location and token type.

## Results: Tokenizer

| Method      | Dense | Moderate | Sparse |
|-------------|-------|----------|--------|
| Old Futhark | 1×    | 1×       | 1×     |
| New Futhark | 1.6×  | 1.8×     | 2×     |

| Method      | Dense | Moderate | Sparse |
|-------------|-------|----------|--------|
| New Futhark | 1×    | 1×       | 1×     |
| 2K          | 1.8×  | 1.2×     | 1.1×   |

- Simulates DFA.
- Determines location and token type.

# Conclusion

- Review of hand-written optimisations.
- A simple language extension that generalizes fusion.
- High-level programs become competitive to hand-written code.

**Benchmarks**



**Futhark**

